Apuntes de Angular

Angular 17–19 · Signal-First · Survey Maker

Contenido

- Signals (`signal`, `computed`, `effect`)
- Bloque 1 — Componentes (`@Component`, `input`, `output`)
- Bloque 2 — Templates (`@if`, `@for`, `@defer`)
- Bloque 3 — Decoradores (`@Component`, `@Injectable`, `@Pipe`)
- Bloque 4 — Directivas (atributo, estructural, `exportAs`)
- (próximos bloques aquí)

Aldo Zetina — UNAM Ciencias de la Computación

github.com/AldoZM/Survey-Maker

Mayo 2026

Compilar localmente

```
lualatex ApuntesAngular.tex
```

(requiere MiKTeX o TeX Live con LuaLaTeX)

Índice

Parte I

⚡ Signals — El Sistema Reactivo

1. 🧠 La Idea Central — Sin Código

Antes de ver código, entiende el concepto con una historia.

▯ La Pizarra Mágica

Imagina que tienes una **pizarra mágica** en tu cuarto. Escribes un número en ella: *"0 respuestas contestadas"*.
Afuera hay tres amigos mirando por la ventana:

- 📊 **Barra de progreso** — actualiza su dibujo cuando cambia el número.
- ✅ **Botón de Enviar** — se activa cuando el número llega al máximo.
- 💾 **localStorage** — guarda una foto del número automáticamente.

Cada vez que **borras el número y escribes uno nuevo**, los tres amigos lo ven al instante. Tú no tienes que avisarles — ellos solos reaccionan.

Eso es un Signal. La pizarra mágica.

2. ✏️ Tipo 1 — `signal()` — La Pizarra Mágica

2.1. Qué es

Un `signal()` crea una **variable reactiva**. Cuando su valor cambia, Angular actualiza todo lo que depende de él automáticamente.

2.2. 📂 En el proyecto

Archivo: `core/services/survey-state.service.ts`, línea 28.

```
// signal() crea la "pizarra magica".
// Todo empieza vacio {} — nadie ha contestado nada todavia.
// Record<string, FieldAnswer> = diccionario donde:
//   la LLAVE es el ID del campo  (ej: "nombre", "calificacion")
//   el VALOR es la respuesta     (ej: { value: "Juan", isValid: true })
readonly answers = signal<Record<string, FieldAnswer>>({});
//                                                       ^
```

```
//                                    {} = valor inicial, pizarra vacia
```

2.3. Cómo leer y escribir

```
// LEER — siempre con parentesis ()
const todasLasRespuestas = this.answers();
//                                        ^
//                      () = "dame lo que hay en la pizarra ahora"

// ESCRIBIR con .set() — reemplaza TODO el valor
this.answers.set({ nombre: { value: "Juan", isValid: true } });

// ACTUALIZAR con .update() — modifica sin perder lo anterior
this.answers.update(prev => ({
//                      ^
//              prev = lo que habia antes en la pizarra
  ...prev,            // copia todo lo anterior (no lo borres)
  [fieldId]: {        // solo cambia ESTE campo especifico
    value: "nuevo valor",
    isValid: true,
    isDirty: true,
  }
}));
```

Regla de Oro

`this.answers` → objeto Signal (no el valor).
`this.answers()` → el valor real (el diccionario).

Siempre leer con `()`.

3. Tipo 2 — `computed()` — El Amigo Matemático

3.1. Qué es

Un `computed()` es una variable que **se calcula sola** a partir de otros Signals. Cuando cualquier Signal que lee cambia, el `computed` recalcula automáticamente. Tú nunca tienes que llamarlo.

Celda de Excel con Fórmula

Imagina `C1 = A1 + B1` en Excel.
Cuando cambias `A1` o `B1`, la celda `C1` se actualiza sola.

`progress` es `C1`. `answers` es `A1`.

3.2. 📊 Computed de Progreso

Archivo: core/services/survey-state.service.ts, línea 43.

```
readonly progress = computed(() => {
  const s = this.schema();
  if (!s) return 0;

  // Aplanamos todas las secciones a un solo array de campos
  const allFields = s.sections.flatMap(sec => sec.fields);
  const requiredFields = allFields.filter(f => f.validation?.required);
  if (requiredFields.length === 0) return 100;

  // Un campo cuenta SOLO si: isValid=true Y isDirty=true
  const answeredRequired = requiredFields.filter(f => {
    const ans = this.answers()[f.id];
    //                ^
    //   LEE "answers" Signal -> crea la dependencia magica.
    //   Si "answers" cambia -> este computed recalcula solo.
    return ans?.isValid && ans.isDirty;
  });

  // (contestados / total) x 100
  return Math.round((answeredRequired.length / requiredFields.length) *
      100);
});
```

3.3. ✅ Computed de Validez

Archivo: core/services/survey-state.service.ts, línea 72.

```
// ?puede el usuario enviar la encuesta?
readonly isValid = computed(() =>
  // .every() = "todos los campos cumplen la condicion?"
  // Si UN SOLO campo falla -> regresa false -> boton bloqueado
  this.schema()?.sections
    .flatMap(s => s.fields)
    .every(f => {
      if (!f.validation?.required) return true; // opcional -> valido
      const ans = this.answers()[f.id]; // dependencia magica
      return ans?.isValid ?? false;
    }) ?? false
);
```

4. Tipo 3 — `effect()` — El Guardián Automático

4.1. Qué es

Un `effect()` ejecuta código cada vez que un Signal cambia. No retorna nada — solo realiza **efectos secundarios**: guardar datos, hacer log, llamar APIs.

La Alarma Automática

La configuras una vez y te olvidas.
Se activa sola cada vez que el Signal cambia.

En el proyecto: *"cada vez que `answers` cambie, guárdalo en localStorage".*

Archivo: `core/services/survey-state.service.ts`, línea 88.

```
effect(() => {
  // untracked() = "lee este Signal pero NO lo vigiles"
  // Solo queremos reaccionar cuando ANSWERS cambia, no cuando schema cambia
  const s = untracked(() => this.schema());

  const answers = this.answers(); // ESTA linea crea la dependencia

  if (!s) return;
  try {
    // Autoguardado — como Google Docs guardando tu documento
    localStorage.setItem(`survey-state-${s.id}`, JSON.stringify(answers));
  } catch {
    // localStorage puede fallar en modo privado o si esta lleno
  }
}, { injector: this.injector });
```

5. Tipo 4 — `computed()` para Lógica Condicional

Decide si un campo se muestra según la respuesta de otro campo. Archivo: `features/survey/field-renderer.component.ts`, línea 156.

```
// "el campo de comentarios solo aparece si elegiste 'Malo'"
readonly isVisible = computed(() => {
  const condition = this.field().condition;
  if (!condition) return true; // sin condicion -> siempre visible

  const answer = this.answers()[condition.dependsOn];
  const expected = condition.equals;
```

```

  // equals puede ser array: ["malo", "regular"] -> cualquiera vale
  if (Array.isArray(expected)) return expected.includes(answer as string);
  return answer === expected;
});
```

No es `display:none`

Cuando `isVisible()` = `false`, Angular **elimina el componente del DOM**. No existe en memoria — cero costo de rendimiento.

6. ⛔ Inmutabilidad — Por Qué Nunca Mutamos Directo

```
// MAL — misma referencia, Angular NO detecta el cambio
prev[fieldId] = nuevoValor;
this.answers.set(prev); // Angular ve el mismo objeto -> no re-renderiza

// BIEN — nueva referencia, Angular SÍ detecta el cambio
this.answers.update(prev => ({
  ...prev,                // copia todos los campos anteriores
  [fieldId]: nuevoValor   // sobreescribe solo el que cambio
}));
```

🏠 Por Qué Importa la Referencia

Angular compara **direcciones de objetos**, no su contenido.
Con `{...prev}` creas un objeto *nuevo* en otra dirección de memoria → Angular detecta el cambio y actualiza la UI.

7. 📋 Resumen — Signals

Tipo	Para qué	Ejemplo en el proyecto
`signal()`	Guardar estado que cambia	`answers`, `schema`, `isSubmitted`
`computed()`	Calcular a partir de Signals	`progress`, `isValid`, `isVisible`
`effect()`	Efectos secundarios automáticos	Guardar en `localStorage`

Parte II

🧩 Bloque 1 — El Componente a Fondo

8. 🏗️ ¿Qué es un Componente?

Un componente es **la unidad mínima de UI en Angular**. Todo lo que ves en pantalla es un componente — o varios componentes anidados.

El Control Remoto

Un componente tiene:

- Una **carcasa** — el template HTML (lo que el usuario ve)
- **Botones que aceptan instrucciones** — los `input()`
- **Una pantalla que avisa hacia afuera** — los `output()`
- **Lógica interna** — la clase TypeScript

9. 🔬 Anatomía de `@Component`

```
@Component({
  // 1. Nombre del tag HTML que usas en templates
  //    Convencion: siempre "app-" + nombre-en-kebab-case
  selector: 'app-text-field',

  // 2. Sin NgModule (Angular moderno — el componente se autogestiona)
  standalone: true,

  // 3. Que otros componentes/directivas usa SU template
  //    Solo lo que usa ESTE componente, nada mas
  imports: [MatFormFieldModule, MatInputModule],

  // 4. El HTML del componente (inline o en archivo separado)
  template: `
    <mat-form-field>
      <input matInput [value]="value()" />
    </mat-form-field>
  `,

```

```
  // 5. CSS encapsulado — solo aplica a ESTE componente
  //    No choca con estilos de otros componentes
  styles: ['.full-width { width: 100%; }'],
})
export class TextFieldComponent { }
```

10. 📥 `input()` — Recibir Datos del Padre

10.1. Viejo vs Nuevo

```
// VIEJO (Angular < 17) — decorador @Input()
@Input() value: string = '';
@Input({ required: true }) field!: QuestionField;

// NUEVO (Angular 17+) — funcion input()
readonly value = input<string>('');                // con default
readonly field = input.required<QuestionField>(); // obligatorio
```

10.2. Por qué `readonly`

```
// readonly = el componente NO puede cambiar su propio input.
// Solo el PADRE puede cambiar el valor pasandole uno nuevo.
// Si intentas hacer this.field.set(...) -> error de TypeScript.
readonly field = input.required<QuestionField>();
```

10.3. Los tres sabores

```
// 1. Opcional con default
readonly value = input<string>('');

// 2. Obligatorio — TypeScript error si el padre no lo pasa
readonly field = input.required<QuestionField>();

// 3. Con transform — transforma el valor antes de guardarlo
readonly disabled = input<boolean, string>(false, {
  transform: (v) => v === 'true' || v === true
});
```

10.4. Cómo se ve desde el padre

```
<app-field-renderer
  [field]="field"                          <!-- [] = pasar dato al hijo
    -->
```

```
  [answers]="answersAsValues()"
  [value]="state.answers()[field.id]?.value"
  [errorMessage]="state.getError(field.id)"
  (valueChange)="state.setAnswer(field.id, $event)"  <!-- () = escuchar al hijo -->
/>
```

11. 📤 `output()` — Avisar al Padre

11.1. Viejo vs Nuevo

```
// VIEJO — @Output() con EventEmitter
@Output() valueChange = new EventEmitter<string>();
this.valueChange.emit('nuevo valor');

// NUEVO — output()
readonly valueChange = output<string>();
this.valueChange.emit('nuevo valor');
```

11.2. En `text-field.component.ts`

```
export class TextFieldComponent {
  readonly field = input.required<QuestionField>(); // recibe del padre
  readonly value = input<string>('');               // recibe del padre
  readonly valueChange = output<string>();           // avisa al padre

  onInput(event: Event): void {
    // Cada vez que el usuario escribe, avisamos al padre
    // $event en el template = lo que emitimos aqui
    this.valueChange.emit((event.target as HTMLInputElement).value);
  }
}
```

📌 Regla de los Corchetes

Datos bajan con `[ ]` — del padre al hijo.
Eventos suben con `( )` — del hijo al padre.

Juntos: `[( )]` = two-way binding (los dos en uno).

12. 💉 `inject()` — Dependency Injection Moderna

12.1. Qué es

🔌 El Enchufe Central

Angular tiene un "enchufe central" con todos los servicios disponibles.
`inject()` es el cable que conecta tu componente a ese enchufe.
Tú no fabricas el servicio — Angular te lo provee listo para usar.

12.2. Viejo vs Nuevo

```
// VIEJO — constructor injection
export class SurveyComponent {
  constructor(
    private state: SurveyStateService,
    private loader: SurveyLoaderService,
  ) { }
}

// NUEVO — inject() funcional
export class SurveyComponent {
  private readonly state  = inject(SurveyStateService);
  private readonly loader = inject(SurveyLoaderService);
}
```

12.3. `providedIn: 'root'` — El Singleton

```
// Una SOLA instancia en toda la app
@Injectable({ providedIn: 'root' })
export class SurveyStateService { }
```

```
// Todos obtienen EL MISMO objeto:
// SurveyComponent       inject(SurveyStateService) -> mismo objeto
// FieldRendererComponent inject(SurveyStateService) -> mismo objeto
// SurveyPageComponent   inject(SurveyStateService) -> mismo objeto

// Por eso cuando SurveyComponent guarda una respuesta,
// FieldRendererComponent la ve inmediatamente.
```

13. ⏳ Lifecycle Hooks — Los Momentos de Vida

```
// NACIMIENTO          VIDA                    MUERTE
// constructor()  ->   ngOnChanges() xN  ->  ngOnDestroy()
// ngOnInit() x1  ->   ngAfterViewInit() x1
```

13.1. `ngOnInit()` — El más usado

```
export class SurveyPageComponent implements OnInit {
  readonly loader = inject(SurveyLoaderService);

  ngOnInit(): void {
    // Corre UNA SOLA VEZ cuando el componente aparece en el DOM.
    // Aqui van: llamadas HTTP iniciales, inicializacion de estado.
    // Los input() YA tienen sus valores en este punto.
    this.loader.loadSurvey('ejemplo-feedback');
  }
}
```

13.2. `ngOnDestroy()` — Limpieza

```
export class MiComponente implements OnDestroy {
  private subscription = interval(1000).subscribe(() => console.log('tick
      '));

  ngOnDestroy(): void {
    // Corre cuando el componente se elimina del DOM.
    // Sin limpieza -> memory leak (la app se come la RAM).
    this.subscription.unsubscribe();
  }
}
```

13.3. `effect()` como alternativa moderna a `ngOnChanges()`

```
// survey.component.ts — forma moderna con Signals
export class SurveyComponent {
  readonly schema = input.required<SurveySchema>();

  constructor() {
    // effect() detecta cuando schema() cambia — igual que ngOnChanges
    // pero con Signals: mas preciso, mas eficiente
    effect(() => {
      const s = this.schema();
      if (s) this.state.initSurvey(s);
    });
  }
}
```

14. `model()` — Two-Way Binding con Signals

```
// Sin model() — necesitas dos propiedades separadas
readonly value       = input<string>('');
readonly valueChange = output<string>();

// Con model() — una sola propiedad, two-way binding automatico
readonly value = model<string>('');

// En el template del padre:
// [(value)]="miVariable"
// Cuando el hijo hace value.set('x') -> miVariable se actualiza sola
```

15. Resumen — Bloque 1

Concepto	Para qué
`@Component`	Define un componente (selector, template, styles, imports)
`input()`	Recibir datos del padre (solo lectura)
`input.required()`	Input obligatorio — error si el padre no lo pasa
`output()`	Avisar al padre con eventos
`model()`	Two-way binding (input + output en uno)
`inject()`	Obtener servicios sin constructor
`providedIn: 'root'`	Singleton — una instancia para toda la app
`ngOnInit()`	Código al nacer (después de que inputs están listos)
`ngOnDestroy()`	Limpieza al morir (subscriptions, timers)
`effect()`	Reaccionar a cambios de inputs (forma moderna)

Parte III

🎨 Bloque 2 — Templates Modernos (Angular 17+)

16. 🧠 ¿Qué es un Template?

El template es el HTML que define **lo que el usuario ve**. Pero no es HTML normal — Angular lo procesa y lo hace dinámico.

🧑‍🍳 La Receta con Instrucciones

Piensa en el template como una **receta con instrucciones condicionales**: *"Si hay huevos, haz omelette. Si no hay, haz cereal. Para cada ingrediente en la lista, agrégalo al plato."*
Eso es exactamente lo que hacen `@if` y `@for`.

17. 💬 Interpolación `{{ }}`

```
<mat-card-title>{{ schema().title }}</mat-card-title>
<!--                ^  Signal — () obligatorios para leer -->

<span>{{ state.progress() }}% completado</span>
<!-- Si progress() = 60 renderiza: "60% completado" -->

<button>{{ schema().submitLabel ?? 'Enviar' }}</button>
<!--                              ^  null/undefined -> 'Enviar' -->
```

18. 🔗 Property Binding `[ ]` y Event Binding `( )`

```
<!-- SIN corchetes — pasa el TEXTO literal "value()" -->
<input value="value()">  <!-- INCORRECTO -->

<!-- CON corchetes — evalua la EXPRESION -->
<input [value]="value()">  <!-- CORRECTO -->

<!-- Event binding — escucha eventos -->
<input (input)="onInput($event)">
<button (click)="onSubmit()">Enviar</button>
```

```
<input (keyup.enter)="buscar()">  <!-- solo al presionar Enter -->

<!-- Output de hijo -->
<app-select-field (valueChange)="state.setAnswer(field.id, $event)" />
```

Regla

`[ ]` → datos bajan del padre al hijo.
`( )` → eventos suben del hijo al padre.

19. `@if` / `@else` — Renderizado Condicional

```
<!-- VIEJO — *ngIf, el else era confuso -->
<div *ngIf="condicion; else miElse">Si</div>
<ng-template #miElse><div>No</div></ng-template>

<!-- NUEVO Angular 17+ -->
@if (condicion) {
  <div>Si</div>
} @else if (otraCondicion) {
  <div>Tal vez</div>
} @else {
  <div>No</div>
}
```

19.1. En `survey-page.component.ts`

```
@if (loader.survey.isLoading()) {
  <mat-progress-bar mode="indeterminate" />
}

@if (loader.survey.error(); as err) {
  <!--                        ^ "as err" captura el valor en variable local
      -->
  <pre>{{ err }}</pre>
}

@if (loader.survey.value(); as schema) {
  <app-survey [schema]="schema" />
}
```

⚠ `@if` NO es `display:none`

`display:none`	`@if`
Elemento **existe** en el DOM	Elemento **no existe**
Angular ejecuta su código	Angular no ejecuta nada
Consume memoria y CPU	Cero costo cuando es `false`

20. 🔁 `@for` — Iteración Reactiva

```
<!-- VIEJO — trackBy era opcional -->
<div *ngFor="let s of sections; trackBy: trackByFn">{{ s.title }}</div>

<!-- NUEVO — track es OBLIGATORIO -->
@for (section of schema().sections; track section.id) {
  <section>
    <h3>{{ section.title }}</h3>
    @for (field of section.fields; track field.id) {
      <app-field-renderer [field]="field" ... />
    }
  </section>
}
```

ID Por qué `track` importa

Sin `track`: Angular destruye y recrea **todos** los elementos al cambiar la lista.
Con `track`: Angular solo destruye **los que desaparecieron**.
Animaciones intactas, focus conservado, inputs sin limpiarse.

20.1. Variables automáticas de `@for`

```
@for (item of lista; track item.id;
      let i = $index, first = $first, last = $last) {
  <!--      ^               ^                ^
            0,1,2...        true si primero   true si ultimo
            Otras: $even, $odd, $count -->
  <div [class.ultimo]="last">{{ i + 1 }}. {{ item.nombre }}</div>
}
```

20.2. `@empty`

```
@for (encuesta of encuestas; track encuesta.id) {
  <tarjeta [data]="encuesta" />
} @empty {
  <p>No hay encuestas disponibles.</p>
}
```

21. ↔️ `@switch` — Despacho por Tipo

```
@let f = field(); <!-- @let = variable local, evita llamar field() 7 veces -->

@switch (f.type) {
  @case ('text')     { <app-text-field     [field]="f" [value]="stringValue()"
                          (valueChange)="valueChange.emit($event)" /> }
  @case ('email')    { <app-text-field     [field]="f" [value]="stringValue()"
                          (valueChange)="valueChange.emit($event)" /> }
  @case ('number')   { <app-number-field   [field]="f" [value]="numberValue()"
                          (valueChange)="valueChange.emit($event)" /> }
  @case ('select')   { <app-select-field   [field]="f" [value]="stringValue()"
                          (valueChange)="valueChange.emit($event)" /> }
  @case ('radio')    { <app-radio-field    [field]="f" [value]="stringValue()"
                          (valueChange)="valueChange.emit($event)" /> }
  @case ('checkbox') { <app-checkbox-field [field]="f" [value]="arrayValue()"
                          (valueChange)="valueChange.emit($event)" /> }
}
```

22. ⏳ `@defer` — Lazy Loading en Template

```
@defer (on interaction) {
  <editor-gigante [contenido]="post" />   <!-- descarga al hacer click -->
} @placeholder {
  <div>Haz clic para editar...</div>      <!-- antes de interactuar -->
} @loading (minimum 300ms) {
  <mat-spinner diameter="32" />           <!-- mientras descarga -->
} @error {
  <p>Error al cargar el editor.</p>       <!-- si la descarga falla -->
}
```

22.1. Triggers

Trigger	Cuándo carga
`on idle`	Cuando el browser no tiene nada que hacer
`on viewport`	Cuando el elemento entra en la pantalla
`on interaction`	Al hacer hover o click
`on hover`	Solo al pasar el mouse encima
`on timer(3s)`	Después de 3 segundos
`when condicion()`	Cuando una expresión Signal es `true`

23. 🧮 Pipes — Transformadores en el Template

```
{{ fecha   | date:'dd/MM/yyyy' }}  <!-- 2026-05-13 -> "13/05/2026" -->
{{ precio  | currency:'MXN'    }}  <!-- 1500 -> "MX$1,500.00" -->
{{ texto   | uppercase         }}  <!-- "hola" -> "HOLA" -->
{{ objeto  | json              }}  <!-- { id:1 } -> '{"id":1}' (debug) -->

<!-- Pipes encadenados izquierda a derecha -->
{{ nombre | uppercase | slice:0:10 }}  <!-- "Aldo Zetina" -> "ALDO ZETIN" -->
```

24. Resumen — Bloque 2

Sintaxis	Para qué
`{{ expr }}`	Mostrar texto (interpolación)
`[prop]`	Pasar dato a elemento o componente hijo
`(evento)`	Escuchar evento de elemento o componente hijo
`[(modelo)]`	Two-way binding
`@if / @else`	Renderizar si condición es `true`
`@for + track`	Iterar lista de forma eficiente
`@empty`	Cuando la lista está vacía
`@switch / @case`	Seleccionar según valor
`@let var = expr`	Variable local en template
`@defer (trigger)`	Carga diferida de componentes pesados
`pipe | nombre`	Transformar valor sin modificar el original

Parte IV

Bloque 3 — Decoradores

25. ¿Qué es un Decorador?

Un decorador es una función que **agrega comportamiento** a una clase, método o propiedad sin modificar su código interno. Se escribe con `@`.

```
// Sin decorador — clase normal, Angular no sabe que es
class TextFieldComponent { }

// Con decorador — Angular sabe que es un componente UI
@Component({ selector: 'app-text-field', ... })
class TextFieldComponent { }
```

La Etiqueta del Producto

Un decorador es como una **etiqueta en un producto**.
La camiseta es la misma, pero la etiqueta le dice al sistema:
"frágil", "lavar a mano", "exportación".

Angular lee esas etiquetas para saber qué hacer con cada clase.

26. `@Component` — "Esto es una pieza de UI"

Ya visto en Bloque 1. Adición importante: `ChangeDetectionStrategy.OnPush`.

```
@Component({
  selector:        'app-survey',
  standalone:      true,
  imports:         [...],
  template:        `...`,
  styles:          [`...`],
  changeDetection: ChangeDetectionStrategy.OnPush, // optimizacion
})
export class SurveyComponent { }
```

```
// Sin OnPush: Angular revisa TODOS los componentes en cada evento.
// Con OnPush solo revisa cuando:
//   1. Un input() cambia de referencia
//   2. Un Signal dentro del componente cambia
//   3. Un evento ocurre DENTRO del componente
```

```
// Perfecto para componentes que usan Signals — como todo tu proyecto.
```

El Modo Turbo

Sin OnPush = revisar **todas** las habitaciones del hotel cada vez que alguien toca un timbre.
Con OnPush = solo revisar la habitación de **donde vino** el timbre.

27. `@Injectable` — "Esto es un servicio"

Le dice a Angular que esta clase puede ser **inyectada** en otros componentes o servicios.

```
// survey-state.service.ts linea 23
@Injectable({ providedIn: 'root' })
export class SurveyStateService {
  // providedIn: 'root' = una sola instancia en toda la app (singleton)
  // providedIn: 'any'  = una instancia por lazy-loaded module
  // Sin providedIn     = debes registrarlo manualmente en providers: []
}
```

27.1. Los niveles de `providedIn`

```
// 1. Root — singleton global (el mas comun)
@Injectable({ providedIn: 'root' })
export class AuthService { }
// Una instancia. Todos los componentes comparten el mismo objeto.

// 2. A nivel de componente — instancia nueva por cada componente
@Component({
  providers: [FormStateService]
  // cada <app-form> crea SU PROPIA instancia de FormStateService
  // al destruirse el componente, el servicio se destruye tambien
})
export class FormComponent { }
```

La Cocina del Restaurante

`providedIn: 'root'` = cocina central. Todos los meseros (componentes) van a la misma.
`providers: [Servicio]` en componente = cada mesero tiene su propia cocina.
La mayoría de casos: cocina central.

28. `@Directive` — "Esto modifica elementos HTML"

Agrega comportamiento a elementos HTML existentes **sin crear nuevo HTML**.

```
// shared/directives/required-mark.directive.ts
@Directive({
  selector: '[appRequiredMark]', // aplica a cualquier elemento con este
      atributo
  standalone: true,
})
export class RequiredMarkDirective implements OnInit {
  readonly appRequiredMark = input<boolean>(false);

  constructor(
    private readonly el: ElementRef<HTMLElement>, // referencia al
        elemento DOM
    private readonly renderer: Renderer2,          // herramienta segura
        para el DOM
  ) {}

  ngOnInit(): void {
    if (this.appRequiredMark()) {
      const span = this.renderer.createElement('span');
      this.renderer.setProperty(span, 'textContent', ' *');
      this.renderer.setStyle(span, 'color', '#f44336');
      this.renderer.appendChild(this.el.nativeElement, span);
    }
  }
}
```

```
<!-- Uso: el atributo activa la directiva -->
<div [appRequiredMark]="field().validation?.required ?? false">
  {{ field().label }}
</div>
<!-- Si required=true -> el div queda: "Nombre *" con el * en rojo -->
```

28.1. ElementRef vs Renderer2

```
// ElementRef — acceso DIRECTO al nodo DOM nativo
// PELIGROSO: abre la puerta a XSS si insertas contenido del usuario
this.el.nativeElement.style.color = 'red'; // funciona pero riesgoso

// Renderer2 — abstraccion SEGURA sobre el DOM
// RECOMENDADO: funciona en SSR, Web Workers, y es XSS-safe
this.renderer.setStyle(this.el.nativeElement, 'color', 'red');
```

Regla de Seguridad

Siempre `Renderer2` sobre `ElementRef.nativeElement` directo.
Si Angular corre en SSR (servidor), `Renderer2` sigue funcionando.
El acceso directo explota sin DOM.

29. 👂 `@HostListener` — Escuchar Eventos del Host

Escucha eventos del **elemento donde está aplicada** la directiva o componente.

```
@Directive({ selector: '[appClickTracker]', standalone: true })
export class ClickTrackerDirective {

  // Escucha click del elemento host
  @HostListener('click', ['$event'])
  onClick(event: MouseEvent): void {
    console.log('Click en:', event.target);
  }

  // Escucha eventos del DOCUMENTO completo
  @HostListener('document:keydown.escape')
  onEscape(): void {
    console.log('Usuario presiono Escape');
  }

  // Escucha eventos de la VENTANA
  @HostListener('window:resize', ['$event'])
  onResize(event: Event): void {
    console.log('Ventana redimensionada');
  }
}
```

🎤 El Micrófono Pegado al Elemento

`@HostListener` escucha todo lo que pasa en el elemento host
y ejecuta tu código cuando ocurre el evento.

30. 🎨 `@HostBinding` — Bindear al Elemento Host

Conecta una propiedad de tu directiva a un **atributo o clase del elemento host**.

```
@Directive({ selector: '[appResaltado]', standalone: true })
export class ResaltadoDirective {

  @HostBinding('class.activo')        isActivo = false;
  @HostBinding('attr.disabled')       isDeshabilitado = false;
  @HostBinding('style.backgroundColor') color = 'transparent';

  @HostListener('mouseenter')
  onEnter(): void {
    this.isActivo = true;
    this.color = '#e3f2fd'; // azul claro al hacer hover
  }

  @HostListener('mouseleave')
```

```
  onLeave(): void {
    this.isActivo = false;
    this.color = 'transparent';
  }
}
```

```
<!-- Uso: al hacer hover -> class="activo" y fondo azul automatico -->
<div appResaltado>Pasa el mouse aqui</div>
```

31. 🧮 `@Pipe` — Transformadores Reutilizables

```
// Pipe que trunca texto largo con "..."
@Pipe({
  name:       'truncar',    // nombre en el template
  standalone: true,
  pure:       true,         // solo recalcula si el INPUT cambia (recomendado)
})
export class TruncarPipe implements PipeTransform {
  transform(valor: string, limite: number = 100): string {
    if (!valor) return '';
    if (valor.length <= limite) return valor;
    return valor.slice(0, limite) + '...';
  }
}
```

```
{{ "Texto muy largo que necesita truncarse" | truncar:20 }}
<!-- -> "Texto muy largo que " -->

{{ pregunta.label | truncar }}     <!-- default 100 chars -->
{{ pregunta.label | truncar:50 }}  <!-- limite 50 chars -->
```

31.1. `pure` vs `impure`

```
// pure: true (default) — recalcula SOLO cuando el valor de entrada cambia
// Eficiente. Angular cachea el resultado.
@Pipe({ name: 'miPipe', pure: true })

// pure: false — recalcula en CADA ciclo de deteccion de cambios
// Costoso. Solo si el pipe depende de estado externo (fecha actual, etc.)
@Pipe({ name: 'ahora', pure: false })
export class AhoraPipe implements PipeTransform {
  transform(): string {
    return new Date().toLocaleTimeString(); // cambia cada segundo
  }
```

```
}
```

32. 📋 Resumen — Bloque 3

Decorador	Para qué
`@Component`	Pieza de UI con template y estilos
`@Injectable`	Servicio inyectable (`providedIn: 'root'` = singleton)
`@Directive`	Modifica elementos HTML sin crear nuevo HTML
`@Pipe`	Transforma valores en el template
`@HostListener`	Escucha eventos del elemento host o globales
`@HostBinding`	Conecta propiedades internas a atributos del host
`ElementRef`	Referencia al nodo DOM nativo (usar con cuidado)
`Renderer2`	Forma segura de manipular el DOM (preferida siempre)

Parte V

🎩 Bloque 4 — Directivas a Fondo

33. 🧠 🤔 ¿Qué es una Directiva?

Una directiva es una **instrucción que le das al DOM**. No crea su propio HTML — modifica o controla **elementos que ya existen**.

🎩 🎩 El Sombrero Mágico del DOM

Imagina que tienes un botón HTML aburrido y normal.

Tú le pones un **sombrero mágico** (la directiva) encima.
Ese sombrero puede:

- 🎨 Cambiarle el color cuando el mouse pasa encima
- ⛔ Bloquearlo si el usuario no tiene permisos
- ✨ Agregar un asterisco si el campo es requerido
- 👀 Esconder el botón en ciertas condiciones

El botón sigue siendo el mismo. El sombrero le da superpoderes.

34. 📁 3️⃣ Los Tres Tipos de Directivas

🏷️ **Tipo**	❓ Para qué	Ejemplo
🏗️ **Componente**	Crea UI nueva	`<app-survey />`
🎨 **Atributo**	Modifica un elemento	`[appRequiredMark]`
✂️ **Estructural**	Agrega o quita del DOM	`@if`, `@for`

💡 😎 La Regla del Corchete

Los selectores con `[corchetes]` son **directivas de atributo**.
Se usan como atributos HTML: `<div [appMiDirectiva]="valor»`

📌 Si el selector es un nombre normal: `<app-cosa>` = componente.

35. 🔍 📖 Disección de `RequiredMarkDirective` (Proyecto Real)

Esta directiva vive en `shared/directives/required-mark.directive.ts` y agrega un asterisco rojo * a los campos obligatorios.

```
// shared/directives/required-mark.directive.ts
@Directive({
  //          [corchetes] = directiva de ATRIBUTO
  //          Selector con nombre del input para que Angular lo matchee
  selector:   '[appRequiredMark]',
  standalone: true,   // no necesita NgModule (Angular moderno)
})
export class RequiredMarkDirective implements OnInit {

  // El input tiene EL MISMO nombre que el selector!
  // Esto permite: <label [appRequiredMark]="true">Nombre</label>
  // Angular pasa "true" a este input automaticamente
  readonly appRequiredMark = input<boolean>(false);
  //                                        ^
  //                             false = no requerido por defecto

  constructor(
    private readonly el:       ElementRef<HTMLElement>, // el elemento donde esta la directiva
    private readonly renderer: Renderer2,               // herramienta segura para el DOM
  ) {}

  ngOnInit(): void {
    if (this.appRequiredMark()) {         // si el campo ES requerido...
      const span = this.renderer.createElement('span');
      this.renderer.setProperty(span, 'textContent', ' *'); // texto: espacio + asterisco
      this.renderer.setStyle(span, 'color', '#f44336');      // color rojo Material
      this.renderer.appendChild(this.el.nativeElement, span);// lo pega DENTRO del label
    }
    // Resultado visual: "Nombre *" con el * en rojo
  }
}
```

35.1. 👀 ❓ ¿Por qué el input se llama igual que el selector?

```
// Template del padre:
<label [appRequiredMark]="field().validation?.required ?? false">
  {{ field().label }}
</label>

// Angular lee: [appRequiredMark]="true"
// Busca: un input() que se llame "appRequiredMark" en la directiva
// Encuentra: readonly appRequiredMark = input<boolean>(false)
// Pasa: true al input
// Resultado: el label queda "Nombre *" con el * en rojo

// Si NO tienen el mismo nombre, necesitas especificarlo:
readonly esRequerido = input<boolean>(false, { alias: 'appRequiredMark'
    });
```

36. 🏷️ 🎨 Patrones de Selector para `@Directive`

```
// 1. Por ATRIBUTO (el mas comun)
@Directive({ selector: '[appTooltip]' })
// Uso: <button appTooltip="Haz click">OK</button>

// 2. Por ATRIBUTO con VALOR especifico
@Directive({ selector: '[role=button]' })
// Activa solo en: <div role="button">

// 3. Por TAG + ATRIBUTO (mas especifico)
@Directive({ selector: 'input[appAutoFocus]' })
// Solo activa en inputs: <input appAutoFocus>
// NO activa en:          <div appAutoFocus>

// 4. Por CLASE CSS
@Directive({ selector: '.mi-clase-especial' })
// Uso: <div class="mi-clase-especial">

// 5. Multiples selectores (OR — activa con cualquiera)
@Directive({ selector: '[appResaltar], [highlight]' })
```

📌 👌 Convención de Nombres

Siempre prefija tus directivas con `app` (o la sigla de tu empresa).
`[appRequiredMark]`, `[appAutoFocus]`, `[appClickTracker]`

⚠️ Sin prefijo puedes chocar con atributos HTML nativos o librerías de terceros.

37. 🔨 🔧 `ElementRef` vs `Renderer2` — La Pelea Eterna

```
// ==============================================
// OPCION 1: ElementRef — acceso DIRECTO al DOM
// ==============================================
// Funciona. Pero tiene problemas serios:
constructor(private el: ElementRef<HTMLElement>) {}

this.el.nativeElement.style.backgroundColor = 'red'; // funciona
this.el.nativeElement.innerHTML = userInput;          // PELIGROSO: XSS!!
//                                  ^
//          Si "userInput" viene del usuario, puede inyectar <script>

// ==============================================
// OPCION 2: Renderer2 — abstraccion SEGURA
// ==============================================
constructor(private renderer: Renderer2) {}

this.renderer.setStyle(this.el.nativeElement, 'background-color', 'red');
this.renderer.setAttribute(this.el.nativeElement, 'aria-label', 'cerrar');
this.renderer.addClass(this.el.nativeElement, 'activo');
this.renderer.removeClass(this.el.nativeElement, 'deshabilitado');
this.renderer.createElement('span');           // crea elemento
this.renderer.appendChild(padre, hijo);        // inserta en DOM
this.renderer.removeChild(padre, hijo);        // elimina del DOM
```

⚠️ 🚨 Por Qué Renderer2 es Obligatorio

Sin Renderer2:

- 💀 XSS: `innerHTML = userInput` ejecuta scripts del usuario
- 💥 SSR roto: `nativeElement` no existe en servidor
- 🚫 Web Workers: el DOM no existe en hilos de trabajo

Con Renderer2: Angular abstrae el DOM.
Misma directiva, funciona en browser, servidor y workers.

38. 👂 🎤 `@HostListener` — El Micrófono de la Directiva

Escucha eventos del elemento donde está aplicada la directiva.

```
@Directive({ selector: '[appHover]', standalone: true })
export class HoverDirective {
```

```

  constructor(
    private readonly el:       ElementRef<HTMLElement>,
    private readonly renderer: Renderer2,
  ) {}

  // Escucha: cuando el mouse ENTRA al elemento
  @HostListener('mouseenter')
  onMouseEnter(): void {
    this.renderer.setStyle(this.el.nativeElement, 'box-shadow', '0 4px 12
        px rgba(0,0,0,0.2)');
    this.renderer.addClass(this.el.nativeElement, 'hovered');
  }

  // Escucha: cuando el mouse SALE del elemento
  @HostListener('mouseleave')
  onMouseLeave(): void {
    this.renderer.removeStyle(this.el.nativeElement, 'box-shadow');
    this.renderer.removeClass(this.el.nativeElement, 'hovered');
  }

  // Escucha: tecla Escape en el DOCUMENTO completo
  @HostListener('document:keydown.escape')
  onEscape(): void {
    console.log('Usuario presiono Escape en cualquier parte de la pagina
        ');
  }

  // Escucha: click con acceso al evento ($event)
  @HostListener('click', ['$event'])
  onClick(event: MouseEvent): void {
    event.stopPropagation(); // evita que el click suba al padre
    console.log('Click en X:', event.clientX, 'Y:', event.clientY);
  }
}
```

```
<!-- Uso -->
<mat-card appHover>
  Esta tarjeta sube cuando el mouse entra
</mat-card>
```

Los Tres Micrófonos

- `'click'` — escucha solo al **elemento host**
- `'document:keydown'` — escucha al **documento completo**
- `'window:resize'` — escucha a la **ventana entera**

39. 🎨 ▯ `@HostBinding` — El Pincel de la Directiva

Conecta una propiedad de tu directiva a una propiedad del elemento host. Cuando la propiedad cambia, el elemento se actualiza automáticamente.

```
@Directive({ selector: '[appValidationState]', standalone: true })
export class ValidationStateDirective {

  // @HostBinding('clase-css') -> agrega/quita esa clase automaticamente
  @HostBinding('class.campo-valido')      isValido      = false;
  @HostBinding('class.campo-invalido')    isInvalido    = false;
  @HostBinding('class.campo-pendiente')   isPendiente   = true;

  // @HostBinding('attr.aria-invalid') -> accesibilidad
  @HostBinding('attr.aria-invalid')       ariaInvalido  = 'false';

  // @HostBinding('style.borderColor') -> CSS inline
  @HostBinding('style.borderColor')       bordeColor    = 'transparent';

  // Input que el padre controla
  readonly isValid = input<boolean | null>(null);

  constructor() {
    // effect() reacciona cuando isValid() cambia
    effect(() => {
      const v = this.isValid();
      this.isValido     = v === true;
      this.isInvalido   = v === false;
      this.isPendiente  = v === null;
      this.ariaInvalido = v === false ? 'true' : 'false';
      this.bordeColor   = v === true ? '#4caf50' : v === false ? '#f44336' : 'transparent';
      //                                ^verde                    ^rojo                ^transparente
    });
  }
}
```

```
<!-- Uso: el padre pasa el estado de validez -->
<input
  matInput
  [appValidationState]="'''"
  [isValid]="state.isFieldValid(field().id)"
/>
<!-- Resultado: borde verde/rojo automatico segun validez -->
```

40. ✂️ ✂️ Directivas Estructurales — Controlan el DOM

Las directivas estructurales **agregan, quitan o repiten** elementos en el DOM. Angular 17+ las implementa con `@if`, `@for`, `@switch`. Pero puedes crear las tuyas propias con `TemplateRef` y `ViewContainerRef`.

🎞️ ✂️ La Sala de Edición

Imagina que tienes una **sala de cine** y un **editor**:

- 🎞️ `TemplateRef` = la **película** sin proyectar (el HTML en espera)
- 🎥 `ViewContainerRef` = el **proyector** (controla qué se muestra)

El editor decide cuándo proyectar la película, cuántas veces, y en qué orden.

```cpp
// Directiva que muestra el contenido solo si el usuario tiene permiso
@Directive({ selector: '[appSiTienePermiso]', standalone: true })
export class SiTienePermisoDirective {

  // Inyecta la "pelicula" (el HTML que envuelve la directiva)
  private readonly template = inject(TemplateRef<unknown>);

  // Inyecta el "proyector" (donde se va a insertar el HTML)
  private readonly vc       = inject(ViewContainerRef);

  // Input: los permisos requeridos
  readonly appSiTienePermiso = input.required<string[]>();

  // Servicio que sabe que puede hacer el usuario actual
  private readonly auth = inject(AuthService);

  constructor() {
    effect(() => {
      const permisosRequeridos = this.appSiTienePermiso();
      const tienePermiso = permisosRequeridos.every(
        p => this.auth.userPermisos().includes(p)
        //    ^  Signal del servicio de autenticacion
      );

      this.vc.clear(); // borra lo que haya en el proyector

      if (tienePermiso) {
        this.vc.createEmbeddedView(this.template);
        // ^ "proyecta la pelicula" = inserta el HTML en el DOM
      }
      // Si NO tiene permiso: vc.clear() y nada mas — el HTML no existe
    });
  }
```

```
}
```

```
<!-- Uso en template -->
<button *appSiTienePermiso="['admin', 'editor']">
  Editar Encuesta
</button>
<!-- Si el usuario no es admin+editor: el boton NO existe en el DOM -->
<!-- No es display:none — literalmente no existe -->
```

*** vs [ ] en Directivas**

`*appDirectiva` = directiva **estructural** (usa `ng-template` internamente)
`[appDirectiva]` = directiva de **atributo** (modifica el elemento existente)

La `*` es azúcar sintáctica: `*appSiTienePermiso="perms"` es igual a: `<ng-template [appSiTienePermiso]="perms»<button>...</button></ng-template>`

41. `exportAs` — Directivas con Nombre

`exportAs` te permite obtener **una referencia a la directiva** desde el template para llamar sus métodos o leer sus propiedades directamente.

```
// Directiva de tooltip que quieres controlar desde el template
@Directive({
  selector:  '[appTooltip]',
  standalone: true,
  exportAs:  'appTooltip',    // el nombre que usaras en el template con
      #
})
export class TooltipDirective {
  private visible = signal(false);

  // Metodos publicos que el template puede llamar
  mostrar(): void { this.visible.set(true); }
  ocultar(): void { this.visible.set(false); }
  toggle():  void { this.visible.update(v => !v); }
  estaVisible(): boolean { return this.visible(); }
}
```

```
<!-- exportAs te da la instancia de la directiva via # -->
<span appTooltip="Haz click aqui" #miTooltip="appTooltip">
  Texto con tooltip
</span>

<!-- Ahora puedes llamar metodos de la directiva directamente -->
<button (click)="miTooltip.toggle()">Mostrar/Ocultar Tooltip</button>
<button (mouseenter)="miTooltip.mostrar()" (mouseleave)="miTooltip.
    ocultar()">
```

```
  Hover me
</button>

<!-- También puedes leer propiedades -->
<p>¿Tooltip visible? {{ miTooltip.estaVisible() }}</p>
```

El Nombre en la Gafete

`exportAs: 'appTooltip'` es como ponerle un **gafete** a la directiva.

Con `#miTooltip=.ªppTooltip"` el template la reconoce por su gafete y puede hablarle directamente: `miTooltip.mostrar()`.

42. `@ContentChild` — Ver lo que Proyecta el Padre

`@ContentChild` permite que un componente acceda a **elementos que el padre insertó** dentro de él mediante `<ng-content>`.

```
// Componente card que expone acceso a su contenido proyectado
@Component({
  selector:  'app-card',
  standalone: true,
  template: `
    <div class="card">
      <div class="card-header">
        <ng-content select="[cardTitle]" />  <!-- slot para el titulo -->
      </div>
      <div class="card-body">
        <ng-content />                       <!-- slot para el resto -->
      </div>
    </div>
  `,
})
export class CardComponent {

  // Obtiene referencia al elemento con atributo "cardTitle"
  // que el padre proyecto dentro de este componente
  @ContentChild('cardTitle') titulo?: ElementRef;

  ngAfterContentInit(): void {
    // En este hook el contenido proyectado YA EXISTE
    if (this.titulo) {
      console.log('El titulo es:', this.titulo.nativeElement.textContent)
          ;
    }
  }
}
```

```
<!-- Uso desde el padre -->
<app-card>
  <h2 cardTitle #cardTitle>Mi Encuesta</h2>  <!-- se proyecta en el header -->
  <p>Descripcion de la encuesta...</p>        <!-- va al body -->
</app-card>
```

El Vigilante de Entrada

`@ContentChild` es como el **vigilante de un edificio**:
sabe exactamente quién entró por la puerta (`<ng-content>`)
y puede hablar con ellos, aunque él no los trajo.

Usa `ngAfterContentInit()` — el contenido proyectado existe solo después de este hook.

43. Caso Práctico — `FieldErrorDirective` Completa

Aplicación real de todo lo que vimos: directiva que muestra mensajes de error bajo un campo de formulario, reactiva a cambios del estado.

```
// shared/directives/field-error.directive.ts
@Directive({
  selector:   '[appFieldError]',
  standalone:  true,
  exportAs:   'appFieldError',    // accesible desde template si se necesita
})
export class FieldErrorDirective {

  // Input: el mensaje de error (null = sin error)
  readonly appFieldError = input<string | null>(null);

  // HostBinding: agrega clase al elemento host automaticamente
  @HostBinding('class.campo-con-error')
  get tienError(): boolean { return this.appFieldError() !== null; }

  // Referencias para crear el span de error en el DOM
  private readonly el       = inject(ElementRef<HTMLElement>);
  private readonly renderer = inject(Renderer2);

  private spanError: HTMLElement | null = null;

  constructor() {
    // effect() reacciona cada vez que appFieldError() cambia
    effect(() => {
      const mensaje = this.appFieldError();
```

```

      if (mensaje) {
        if (!this.spanError) {
          // Primera vez: crea el span y lo inserta despues del campo
          this.spanError = this.renderer.createElement('span');
          this.renderer.addClass(this.spanError, 'error-message');
          this.renderer.setStyle(this.spanError, 'color', '#f44336');
          this.renderer.setStyle(this.spanError, 'fontSize', '12px');
          this.renderer.setStyle(this.spanError, 'display', 'block');
          this.renderer.setStyle(this.spanError, 'marginTop', '4px');
          // insertAfter: inserta el span DESPUES del campo en el DOM
          const padre = this.renderer.parentNode(this.el.nativeElement);
          this.renderer.insertBefore(padre, this.spanError,
            this.el.nativeElement.nextSibling);
        }
        // Actualiza el texto del mensaje
        this.renderer.setProperty(this.spanError, 'textContent', mensaje)
            ;
      } else if (this.spanError) {
        // Sin error: elimina el span del DOM
        const padre = this.renderer.parentNode(this.spanError);
        this.renderer.removeChild(padre, this.spanError);
        this.spanError = null;
      }
    });
  }
}
```

```
<!-- Uso en el template del formulario -->
<input
  matInput
  type="text"
  [appFieldError]="state.getError(field().id)"
  [(ngModel)]="valor"
/>
<!-- Si getError() devuelve "Campo requerido":
     - El input recibe la clase CSS "campo-con-error"
     - Aparece un <span> rojo debajo: "Campo requerido"
     Si getError() devuelve null:
     - La clase desaparece
     - El span se elimina del DOM -->
```

44. 📋 ⭐ Resumen — Bloque 4

🏷️ Concepto	📝 Para qué
`@Directive` atributo	Modifica un elemento HTML existente
`@Directive` estructural	Agrega o quita elementos del DOM
`ElementRef`	Referencia al nodo DOM (con cuidado ⚠️)
`Renderer2`	Manipulación segura del DOM (✅ siempre preferir)
`@HostListener`	Escucha eventos del host o globales
`@HostBinding`	Conecta propiedades a clases/atributos del host
`TemplateRef`	La "película" (HTML sin renderizar)
`ViewContainerRef`	El "proyector" (inserta/quita del DOM)
`exportAs`	Da nombre a la directiva para accederla con `#ref`
`@ContentChild`	Accede a contenido proyectado por el padre

🎓 🏆 Cuándo Usar Cada Tipo

- 🏗️ **Componente**: cuando necesitas HTML nuevo (`<app-algo>`)
- 🎨 **Directiva de atributo**: cuando modificas un elemento que ya existe
- ✂️ **Directiva estructural**: cuando controlas si algo existe en el DOM

💡 La mayoría de casos: directiva de atributo. Las estructurales son raras.